AI for Good Sandbox Hackathon – GM of AI – CascadeGuard

Team leader: Ibrahim Melhem (Al Balqa Applied University; mentor: Dr. Wafaa Almaaitah)
Team members: Ibrahim Melhem
Contact details: ibrahimmelhem2005@gmail.com
Github link: https://github.com/hemobemoo/CascadeGuard

1. Introduction

CascadeGuard finds where AI decisions and institutional rules collide, identifies the smallest safe repair, and replays the same incident to test whether the repair works. Six institutional roles are played by agents (scripted, or a live LLM); a deterministic engine, never an agent, accepts or rejects every proposed action against the loaded clause set: reviewed clauses quoted verbatim from UAE (in English translation), international and foreign documents, and two illustrative scenario rules. It is decision support, not legal interpretation: every diagnosed failure is a candidate gap for expert review, not a confirmed legal or standards gap. The scenario is hypothetical and all data is synthetic.

2. Description of the use case

A telecom AI traffic-management model, newly updated, deprioritises authentication SMS (T+00). Patients retry their one-time passwords; the health portal's AI fraud-risk model reads the retries as credential stuffing and locks 4,812 accounts (T+15). Each operator sees only its own side (T+30); six roles must decide what to report, share and authorise (T+45); 3,950 patients are still locked out, the share highest among SMS-only users and users over 65 (T+60); the window closes at T+75.

Gaps in existing approaches. The incident crosses two AI systems in two sectors, so a review of either organisation alone sees only half of it. In this hypothetical scenario the health operator needs the network status, but the scenario places it outside the operator's information boundary; no clause in the loaded clause set authorises that share, assigns an owner for reversing the automated lockouts, or triggers a review of the automated lockouts (which may stem from another organisation's AI error). Each is a candidate gap for expert review, not a confirmed legal or standards gap.

How CascadeGuard addresses them. Facts are released to each role only within its boundary, at the time it would learn them. The engine checks every action against the loaded clauses, then diagnoses the coordination failures that block recovery goals (information deadlock, orphan duty, undefined trigger). It proposes repairs from a closed grammar of five edit types (assign owner, add escalation trigger, define minimum data-share, add fallback-channel rule, designate decision-requester), discards any that would lift a prohibition or safeguard, and ranks the rest by replaying the identical incident. Repair edits are hypothetical rules, not source clauses.

3. Use case requirements

- **SRC:** timed injects with facts; each fact names its holder and each inject its recipients; synthetic data only.
- **C:** each role receives only the facts inside its information boundary (scenario data), at the inject that releases them.
- **PP:** each view is read-only and boundary-filtered; each role sees only the clauses it may cite, every clause with its status label.
- **M:** an agent returns only a proposal (actions, rationale, citations, uncertainty); it never changes state. A live LLM runs at temperature 0 through a decision cache, so a replay reuses every unchanged step.

- **P:** an action is applied only if it passes the role's permissions, the information boundaries, cited evidence (loaded clause IDs only: reviewed or illustrative, never draft) and the clauses' authority, prohibitions and safeguards. A suspected legal gap counts only as a cited, qualified claim.
- **D:** accepted actions are applied and their facts released; obligations open, are fulfilled or missed.
- **SINK:** recovery goals, obligations and diagnosed failures, each failure with its qualification, in the CLI, UI, JSON/Markdown exports and an audit file.
- **MLFO and sandbox:** a deterministic orchestrator (not an agent) runs the timeline; repairs are tested only in replay, never applied to the baseline.

4. Mapping with ITU-T Y.3172, the AI Readiness 2.0 Report, and domain-specific policies

Y.3172 is used as an architecture pattern for an institutional decision pipeline; CascadeGuard does not deploy ML models into a network. Four of the five Readiness 2.0 dimensions the submission audit supports are claimed (AI for Inclusion, partial, is not).

Node	CascadeGuard element	Readiness 2.0 dimension	Policy documents / clause IDs
SRC	Scenario injects and facts, T+00 to T+75 (synthetic)	Cross-domain correlation analysis	— (synthetic scenario)
C	Per-inject collection of the facts each role may see (`RoleView`)	—	— (information boundaries are scenario data)
PP	Boundary filtering and role-scoped clause retrieval	—	`uae.fl2-2019.art2`, `uae.pdpl.art2-2` (binding, English translation); `dha.pp11.applicability` (binding unknown): working reading of the applicable regime, not a legal conclusion
M	The six role agents (scripted, or live LLM) returning a proposal	—	`uae.ai-charter.safety` (not binding; context for suspending the traffic model)
P	The deterministic engine: evidence, authority, prohibition and safeguard checks; the repair hard filter	AI & Policies; Impacts of Humans in AI Integration	`uae.fl2-2019.art4`, `uae.fl2-2019.art16` (prohibitions as interpreted, art16 conditional; binding, English translation); `uae.fl2-2019.art6` (binding, English translation); `dha.pp11.4-18-1`, `dha.pp11.4-18-2`, `dha.pp11.4-18-4` (binding unknown); `uae.ai-charter.human-oversight` (not binding); `illus.telecom.incident-report`, `illus.telecom.regulator-information-request` (illustrative, synthetic)
D	Applies accepted actions, releases facts, opens and closes obligations	Strategy Alignment	`illus.telecom.incident-report` (illustrative, synthetic); `dha.pp11.4-18-4` (binding unknown)
SINK	Goals, obligations and diagnosed failures, with qualifications	AI & Policies	Reviewed example mapping (not binding): `oecd.ai-principles.accountability-cooperation`, `oecd.ai-principles.robustness-override`, `unesco.ai-ethics.review-and-correct`, `unesco.ai-ethics.human-oversight`, `uae.ai-charter.human-oversight`
MLFO	Orchestrator and repair search over the closed grammar	AI & Policies	— (no ML-intent input; critical goals stand in for it)
Sandbox	Replay of the identical incident with a candidate repair	AI & Policies; Impacts of Humans in AI Integration	— (repair edits are hypothetical rules, not source clauses)

Every clause is shown with its status label (e.g. "reviewed (interpreted) · binding unknown", "illustrative", "not binding · international precedent"); 21 clauses are loaded (19 reviewed, 2 illustrative), drafts never.

5. Error scenarios

Baseline (scripted agents). The engine diagnoses 4 coordination failures: an information deadlock (network status to the health operator), two orphan duties (`activate_fallback_channel`, `reverse_automated_lockouts`) and an undefined trigger (`review_automated_lockouts`). Both critical goals, `patient_access_restored` and `alert_cause_known_to_health`, are unmet; each failure is a candidate gap for expert review, not a confirmed legal or standards gap.

Run (scripted agents)	`patient_access_restored`	`alert_cause_known_to_health`	Failures
Governed	unmet	unmet	4
Ungoverned (comparison mode, never the result)	met at T+60	unmet	1; 3 masked, cause still present
Governed + rank-1 repair	met at T+60	met at T+30	2 removed; 2 masked, cause still present

Ungoverned (scripted agents). Three actions were applied without the authority the governed engine requires, all at T+60: `reverse_automated_lockouts` (missing_information, no_authority), `activate_fallback_channel` (authority_uncertain) and `issue_determination` (authority_uncertain). `patient_access_restored` is met at T+60, but `alert_cause_known_to_health` stays unmet and the three causes are masked, cause still present.

Governed with the rank-1 repair (scripted agents). Make `health_service_operator` the owner of reversing automated lockouts within 30 minutes of the action's opening condition holding, and define a minimum share of network incident status to `health_service_operator` (from `telecom_operator` or another holder): 2 policy edits, hypothetical rules, not source clauses. All critical goals are met, with the engine's checks in force. 2 failures are removed at their cause; 2 are masked, cause still present: `orphan_duty:activate_fallback_channel` and `undefined_trigger:review_automated_lockouts`. The repair needs human and institutional approval: it creates a permission that no reviewed, binding clause in the loaded clause set supports. Of 21 ranked repairs, 2 meet every critical goal.

Live LLM agents (gate-4 recording, separate evidence). With a live model, the governed run shows the same 4 failures and `patient_access_restored` stays unmet. In the ungoverned live run it is met at T+15, through `activate_fallback_channel` applied without the authority the governed engine requires (authority_uncertain: `dha.pp11.4-18-1`, binding unknown; at T+15, T+60 and T+75). No live repair was run.

Controversy: "governance slows recovery". With scripted agents, the ungoverned run and the rank-1 repair both meet `patient_access_restored` at T+60, with the same scripted decisions: here, in this scripted replay where agents do not adapt, governance with the repair costs no time, and only the repair run also meets `alert_cause_known_to_health`. With live agents, `patient_access_restored` is met at T+15 in the ungoverned run and unmet in the governed run; no live repair was run, so there is no live repair result to set against T+15, and scripted and live numbers are not compared. CascadeGuard's answer: the live speed came from `activate_fallback_channel` applied without the authority the governed engine requires. The engine makes that visible as a diagnosed orphan duty (`orphan_duty:activate_fallback_channel`) and, with scripted agents (not tested with live agents), proposes a repair that restores access with the checks in force; that repair needs human and institutional approval, and two of its blockers remain masked, cause still present.

6. Policy gaps and standards gaps derived from the mapping exercise

Each item is a candidate gap for expert review, not a confirmed legal or standards gap. Examples are from the reviewed mapping (gate 7, 2026-10-31); international and foreign texts are not binding. Repairs are hypothetical rules, not source clauses, tested with scripted agents; each needs human and institutional approval.

- **Information deadlock (network status to the health operator).** Not found in the loaded clause set: no clause authorising this share. Example: `oecd.ai-principles.accountability-cooperation` (co-operation between AI actors, as appropriate). Removed at its cause by the rank-1 repair.

- **Orphan duty: reversing automated lockouts.** Not found in the loaded clause set. Examples: `oecd.ai-principles.robustness-override`, `unesco.ai-ethics.review-and-correct`, `uae.ai-charter.human-oversight` (not binding). Removed at its cause by the rank-1 repair.
- **Orphan duty: activating the fallback channel.** Only uncertain authority was found (`dha.pp11.4-18-1`, binding unknown); it requires a determination. No reviewed example clause is mapped to this failure. The rank-1 repair only masks it: it stays open for the authorities to determine.
- **Undefined trigger: reviewing automated lockouts.** Not found in the loaded clause set. Examples: `unesco.ai-ethics.human-oversight` and `uae.ai-charter.human-oversight`, which state oversight but set no trigger for review. The rank-1 repair only masks it: it stays open for expert review.

The two masked items are the candidate gaps the rank-1 repair leaves open: in the scripted replay the incident resolves without settling who may activate a fallback channel or what triggers a review of an automated lockout. Both need expert review.

Limits. The policy and legal adviser answers only at scenario steps; it cannot be queried live. No fine-tuning and no Sandbox credits were used. Y.3172 is used as an architecture pattern. The two telecom rules (`illus.telecom.*`) are illustrative and synthetic. Reproduce with no API key: `uv sync`, then `uv run cascadeguard run uae_health_access` (see the README).